

PES UNIVERSITY, BANGALORE
Department of Computer Science and Engineering

Pocket Packet Sniffer

Final Project Documentation

Software Requirements Specification (SRS) • Software Architecture and Design (SAD) • Software
Test Plan (STP)

Group 4 - Jackfruit Phase-1 (UE24CS341A)

Ankita S (PES1UG24CS066) • Anusha Gupta (PES1UG24CS073)
Charan M (PES1UG24CS125) • Vaghasiya Akshar Arvindbhai (PES1UG25CS852)

Date: 6 October 2026

Contents

Part I - Software Requirements Specification	3
Revision History	3
1. Introduction	3
2. Overall Description	4
3. External Interface Requirements	5
4. Analysis Models	5
5. System Features	5
6. Other Nonfunctional Requirements	7
7. Other Requirements	9
Appendix A: Glossary	9
Appendix B: Field Layouts	9
Appendix C: Requirement Traceability Matrix	9
Part II - Software Architecture and Design Specification	11
Revision History	11
Approvals	11
1. Introduction	11
2. Document Overview	12
3. Architecture	12
4. Design	15
5. Appendices	20
Part III - Software Test Plan	22
1. Introduction	22
2. Test Strategy	22
3. Test Environment and Tools	22
4. Test Cases	22
5. Requirements-to-Test Coverage	23
6. Entry and Exit Criteria	23
7. Schedule (aligned with Agile sprints)	23
8. Roles and Responsibilities	23
9. Risks and Mitigations	24
10. Defect Management	24

Part I - Software Requirements Specification

Pocket Packet Sniffer - prepared by Group 4, PES University, Bangalore

Version 2.0 | Created 07 Sep 2026, revised 06 Oct 2026

Revision History

Name	Date	Reason For Changes	Version
Ankita S, Anusha Gupta, Charan M, Vaghasiya Akshar Arvindbhai	07-Sep-2026	Initial SRS drafted from approved project synopsis	1.0
Ankita S, Anusha Gupta, Charan M, Vaghasiya Akshar Arvindbhai	06-Oct-2026	Added measurable NFRs and numbered security requirements (aligned with Test Plan v1.0); added SAD/STP references; completed Appendix C traceability (architecture, design and test references).	2.0

1. Introduction

1.1 Purpose

This document specifies the software requirements for the Pocket Packet Sniffer, a compact, portable network traffic monitoring device developed as a Software Engineering mini-project (Jackfruit Phase-1) at PES University, Bangalore, by Group 4. This is Version 2.0 of the SRS, which supersedes Version 1.0 by adding measurable non-functional requirements (NFR-01 to NFR-05), numbered security requirements (SEC-01 to SEC-05) and architecture, design and test references in the traceability matrix. It covers the complete scope of the mini-project as approved in the project synopsis. This SRS describes the requirements for the entire system, not a subsystem.

1.2 Intended Audience and Reading Suggestions

This document is intended for the development team members (Ankita S, Anusha Gupta, Charan M, Vaghasiya Akshar Arvindbhai), the course instructor and teaching assistants evaluating the project, and any future contributor who needs to understand the system's functional and non-functional requirements. Section 2 gives an overall description of the product. Section 3 defines external interface requirements. Section 5 details each system feature with its functional requirements. Section 6 covers non-functional requirements. Readers primarily interested in scope and features should read Sections 1, 2, and 5; readers interested in quality attributes should focus on Section 6.

1.3 Product Scope

The Pocket Packet Sniffer is a compact, battery-friendly device built around a Raspberry Pi (with an ESP32 as an optional auxiliary hardware component), designed to capture live network traffic from a chosen interface, analyse it using the open-source Zeek network security monitoring engine, and present a simplified, human-readable summary through a lightweight Python-based interface. The product is intended to support networking and cybersecurity laboratory sessions, student learning, and small-scale network troubleshooting, allowing a student or junior network technician to get an immediate overview of network activity on a segment without setting up full capture software on a laptop. The system integrates Zeek for protocol parsing and detection rather than reimplementing it; the team's original contribution is the capture pipeline, portable hardware integration, summarisation and alerting logic on top of Zeek, and session management.

1.4 References

- Pocket Packet Sniffer — Synopsis / Project Proposal, Group 4, PES University, Bangalore.
- Software Architecture and Design Specification (SAD) — Pocket Packet Sniffer, Version 1.0, Group 4, 06-Oct-2026.
- Software Test Plan (STP) — Pocket Packet Sniffer, Version 1.0 (Draft), Group 4, 30-09-2026.
- IMP Details on SE Mini-Project Guidelines: Evaluation, Synopsis and SRS 2026, Department of Computer Science and Engineering, PES University, Bangalore (Subject Code: UE24CS341A).
- Zeek Network Security Monitor documentation — <https://zeek.org>

- IEEE SRS Template (Jackfruit Phase-1 format), PES University.

2. Overall Description

2.1 Product Perspective

The Pocket Packet Sniffer is a new, self-contained product; it is not a follow-on to an existing system. It integrates an established open-source analysis engine (Zeek) rather than reimplementing protocol parsing, which is standard practice for a project of this scope. The system consists of four logically separate layers: (1) a capture layer running on Raspberry Pi hardware (with optional ESP32 auxiliary hardware) that acquires raw packets from a network interface; (2) a Zeek analysis layer that parses protocols, generates structured logs, and applies detection scripts; (3) a Python-based control/UI layer that reads Zeek's output and renders a live summary and alerts; and (4) a storage/export layer that persists and exports captured sessions. The device operates standalone at the point of network interest and does not depend on connectivity to any external server.

2.2 Product Functions

At a high level, the product allows a user to:

- Select a network interface and start, stop, or time-bound a live packet capture session, optionally applying a basic filter.
- View a real-time, simplified summary of network activity (protocol breakdown, top talkers, connection counts) while a capture is running or after it ends.
- Enable alerting to have the system flag predefined suspicious traffic patterns (e.g., port-scan behaviour) with timestamp, source, and reason.
- Browse, retrieve, and export past capture sessions (raw .pcap and/or Zeek logs) for offline analysis in tools such as Wireshark.

2.3 User Classes and Characteristics

The primary user class is networking and cybersecurity students and teaching assistants running lab sessions — users with basic networking knowledge but not necessarily prior Wireshark or Zeek expertise. A secondary user class is junior IT or network administrators who need a quick, low-effort way to check traffic on a segment. Both classes are expected to use all functional features of the system; there is no distinct administrator/end-user privilege split, since the device is single-user and operated locally at the point of capture.

2.4 Operating Environment

The system runs on a Raspberry Pi (or equivalent single-board computer) running a Linux-based operating system, with Zeek installed as the packet analysis engine and Python 3 powering the control application and user interface. An ESP32 microcontroller may be used as an auxiliary hardware component where the final hardware design requires additional network interfacing. The device is battery-powered for portability and connects to the target network segment via a wired or wireless network interface. No dependency on cloud services or external servers is required for core operation; local storage (e.g., microSD card) is used to persist capture sessions.

2.5 Design and Implementation Constraints

- Hardware constraint: must run on Raspberry Pi-class hardware with limited CPU, memory, and storage compared to a full laptop/workstation.
- Technology constraint: packet analysis must be performed via Zeek rather than a custom-built protocol parser.
- Language/tooling constraint: the control application and UI must be implemented in Python.
- Data format constraint: raw captures must be stored in standard .pcap format so they remain compatible with external tools such as Wireshark.
- Development process constraint: the project must follow Agile methodology with sprint-based delivery, and use Git/GitHub for version control with active use of a backlog/issue tracker.
- Power/portability constraint: the device must be battery-friendly and self-contained, without requiring a full laptop setup at the point of capture.

- Security constraint: captured traffic data must be stored locally by default with no automatic transmission off the device.

2.6 Assumptions and Dependencies

- It is assumed that Zeek is available and correctly installable on the chosen Raspberry Pi hardware and Linux distribution.
- It is assumed that the network interface used for capture supports promiscuous/monitor mode as required for packet capture on the target segment.
- The project depends on Zeek's built-in and extended detection scripts for identifying suspicious traffic patterns (e.g., port-scan detection); custom signature development beyond what Zeek provides is out of scope.
- It is assumed that capture activity is performed with appropriate authorisation on networks the team is permitted to monitor (e.g., lab environments).
- The project assumes availability of sample and lab-generated traffic for testing, since some detection scenarios (e.g., port scans) cannot be reliably tested against production networks.

3. External Interface Requirements

3.1 User Interfaces

The system exposes a lightweight Python-based interface (terminal-based or simple GUI/web dashboard, depending on final design) providing: a capture control screen to select an interface and start/stop/filter a session; a live-updating dashboard showing protocol breakdown, top talkers, and connection counts; an alerts view listing flagged suspicious events with timestamp, source, and reason; and a session browser for viewing, retrieving, and exporting past sessions. The interface is designed to be understandable to a user without prior Zeek or Wireshark expertise, favouring simplified, readable summaries over raw log output.

3.2 Software Interfaces

The system interfaces with Zeek (the network security monitoring engine), which produces structured log files such as conn.log, dns.log, and http.log. The Python control application reads these logs in real time to generate summaries and alerts. The capture layer interfaces with the underlying operating system's packet capture facilities (e.g., libpcap) to acquire raw traffic and write .pcap files. Exported sessions are formatted for compatibility with external analysis tools such as Wireshark. Local storage uses a file-based or lightweight database structure to index session metadata.

3.3 Communications Interfaces

The device captures traffic over the network interface connected to the target segment (wired Ethernet or Wi-Fi, depending on hardware configuration). No mandatory external network communication (e.g., cloud upload) is required for core functionality, consistent with the requirement that captured data be stored locally by default. Where a future remote-dashboard capability is implemented (see Future Scope), communication would occur over the local network only.

4. Analysis Models

The overall data flow follows a linear pipeline: Network Interface → Capture Layer (Raspberry Pi / optional ESP32) → Zeek Analysis Engine (protocol parsing, structured logs, detection scripts) → Python Control Application (aggregation, summary rendering, alert generation) → Local Storage (session index, .pcap and Zeek logs) → Export (to external tools such as Wireshark). Each of the four functional features described in Section 5 maps to one stage or cross-cutting concern of this pipeline, and each is owned by one team member as described in the project's product ownership plan. The component structure, sequence diagrams and interfaces realising this pipeline are specified in the Architecture and Design Specification (SAD) v1.0.

5. System Features

5.1 Live Capture & Session Control

5.1.1 Description and Priority

Allows the user to select a network interface and start, stop, or time-bound a live packet capture session, optionally applying a basic protocol filter. Priority: High — this is the entry point of the entire pipeline and all other features depend on it. Owner: Ankita S.

5.1.2 Stimulus/Response Sequences

- User selects a network interface and optionally sets a duration or protocol filter → system validates the interface is available and ready.
- User starts a capture → system begins streaming packets into Zeek in real time and writes a raw .pcap file to local storage.
- User stops a capture (or the set duration elapses) → system cleanly terminates the capture and finalises the .pcap file and associated Zeek logs.

5.1.3 Functional Requirements

REQ-1.1: The system shall allow the user to select an available network interface from a list before starting a capture.

REQ-1.2: The system shall allow the user to start and stop a capture session on demand.

REQ-1.3: The system shall allow the user to optionally set a maximum capture duration, after which the session ends automatically.

REQ-1.4: The system shall allow the user to optionally apply a basic filter (e.g., by protocol) to the capture.

REQ-1.5: The system shall stream captured packets to Zeek in real time for the duration of the session.

REQ-1.6: The system shall write the raw packet capture to local storage in standard .pcap format.

REQ-1.7: The system shall handle interface disconnects during a capture gracefully, without crashing the device, and shall notify the user of the interruption.

5.2 Real-Time Traffic Summary Dashboard

5.2.1 Description and Priority

Continuously reads Zeek's structured logs and presents a simplified, live-updating summary of network activity through the Python interface. Priority: High — this is the primary way a user derives value from a capture without reading raw Zeek logs. Owner: Anusha Gupta.

5.2.2 Stimulus/Response Sequences

- A capture session is active or has just ended → system continuously reads Zeek's structured logs (conn.log, dns.log, http.log, etc.).
- New log entries are detected → system aggregates statistics (protocol breakdown, top talkers, connection counts) and updates the summary view.
- User views the dashboard at any point during or after a capture → system displays the current aggregated summary.

5.2.3 Functional Requirements

REQ-2.1: The system shall continuously read Zeek's structured log output while a capture session is active.

REQ-2.2: The system shall compute and display a protocol breakdown of observed traffic.

REQ-2.3: The system shall compute and display a list of top talkers (hosts generating the most traffic or connections).

REQ-2.4: The system shall compute and display connection counts aggregated over the session.

REQ-2.5: The system shall update the displayed summary in real time (or at short, regular intervals) while a capture is running.

REQ-2.6: The system shall present the summary in a simplified textual/graphical form understandable without prior Zeek or Wireshark expertise.

5.3 Anomaly & Suspicious-Activity Alerting

5.3.1 Description and Priority

Uses Zeek's detection capabilities, together with selected project-specific logic, to flag predefined suspicious traffic patterns (e.g., port-scan behaviour) and present them as a prioritised, readable alert list. Priority: High — this differentiates the product from a plain traffic monitor. Owner: Charan M.

5.3.2 Stimulus/Response Sequences

- User enables alerting for a session → system activates the relevant Zeek detection scripts (standard and/or extended).
- Zeek or the project-specific detection logic identifies a suspicious pattern → system generates an alert entry with a timestamp, source, and reason.
- User reviews the alert list → system displays alerts in a prioritised, human-readable format.

5.3.3 Functional Requirements

REQ-3.1: The system shall allow the user to enable or disable alerting for a capture session.

REQ-3.2: The system shall use Zeek's standard and/or extended detection scripts to identify predefined suspicious traffic patterns, including port-scan behaviour.

REQ-3.3: The system shall generate an alert entry containing a timestamp, source, and reason whenever a suspicious pattern is detected.

REQ-3.4: The system shall present alerts to the user in a prioritised, readable list rather than requiring manual inspection of raw connections.

REQ-3.5: The system shall restrict alerting to patterns already supported by Zeek's standard/extended scripts or defined project-specific detection logic; it shall not claim to detect arbitrary or novel attack types outside this scope.

5.4 Session History, Storage & Export

5.4.1 Description and Priority

Maintains a local index of past capture sessions and allows the user to browse, select, and export a session (as .pcap and/or Zeek logs) for offline analysis. Priority: Medium-High — required for the system to be useful beyond a single live session. Owner: Vaghasiya Akshar Arvindbhai.

5.4.2 Stimulus/Response Sequences

- A capture session ends → system adds the session and its associated files/metadata to the local session index.
- User browses past sessions → system displays the list of stored sessions with relevant metadata (e.g., date, duration, interface).
- User selects a session and requests export → system packages the .pcap and/or Zeek logs for that session for offline use.

5.4.3 Functional Requirements

REQ-4.1: The system shall maintain a local index of all completed capture sessions and their associated files.

REQ-4.2: The system shall allow the user to browse and select any past capture session from the index.

REQ-4.3: The system shall allow the user to export a selected session's raw .pcap file and/or Zeek logs for offline analysis.

REQ-4.4: The system shall retain session data between application restarts and device reboots without data loss.

REQ-4.5: Exported .pcap files shall remain compatible with external analysis tools such as Wireshark.

6. Other Nonfunctional Requirements

6.1 Performance Requirements

The capture and log-processing pipeline shall keep up with live traffic on a typical small LAN/lab network segment without noticeable lag in the summary dashboard. The dashboard shall refresh at short, regular intervals during an active capture so that displayed statistics remain representative of current activity.

NFR-01: During an active capture the dashboard shall refresh at least every 2 seconds and reflect new Zeek log entries within 2 seconds.

NFR-02: The pipeline shall process a replayed 5 Mbps traffic stream on Raspberry Pi hardware with no more than 1% of packets dropped.

6.2 Safety Requirements

A capture session shall not crash the device, and the system shall handle network interface disconnects gracefully rather than entering an unrecoverable state. As the device may be used in shared lab environments, the system shall avoid actions (such as active probing) that could disrupt the network segment being monitored, restricting itself to passive capture and analysis.

NFR-03: On an interface disconnect during capture, the application shall not crash and shall notify the user within 5 seconds; the partial .pcap shall remain readable.

6.3 Security Requirements

Captured traffic may include sensitive data; it shall be stored locally by default with no automatic transmission off the device. A basic security review of the alerting logic and data handling shall be conducted by the team as a shared responsibility. Access to stored sessions is assumed to be controlled by physical/OS-level access to the device, since the system is single-user and does not implement its own authentication layer in this phase.

SEC-01: Captured data shall be stored locally and the application shall make no outbound network connection carrying captured data.

SEC-02: Session files (.pcap, Zeek logs, session index) shall be readable only by the owning OS user (permissions 600/700 or stricter).

SEC-03: All user-supplied values (interface name, capture filter, duration) shall be validated and never passed unsanitised to a shell or to Zeek.

SEC-04: The device shall operate passively and shall transmit no probe or injected packets on the monitored segment.

SEC-05: Exports shall be written only to the destination the user selects, with session names sanitised against path traversal.

6.4 Software Quality Attributes

- Usability: the dashboard and alerts shall be understandable to a user without prior Zeek or Wireshark expertise. (NFR-05: a user with no Zeek or Wireshark experience shall be able to start a capture and find the top talker within 5 minutes using only on-screen guidance.)
- Reliability: a capture session shall not crash the device and shall handle interface disconnects gracefully.
- Maintainability: there shall be a clear separation between the capture, analysis, UI, and storage layers, so each can be modified independently.
- Testability: each module shall be testable independently using pre-recorded .pcap files as input, without requiring live network hardware for every test run. (NFR-04: this applies to every module, tested via pre-recorded .pcap replay.)
- Portability: the system shall run on Raspberry Pi-class hardware and shall not depend on proprietary or non-portable components beyond Zeek and Python.

6.5 Business Rules

- The system is intended for use on network segments the user is authorised to monitor (e.g., lab environments); it is not intended for unauthorised network surveillance.
- Only one capture session may be active on a given interface at a time in this phase of the project (multiple simultaneous interfaces are listed under Future Scope).

- Alerting is limited to suspicious patterns supported by Zeek's standard/extended scripts or explicitly defined project-specific logic; the system shall not be represented as a comprehensive intrusion detection or SIEM replacement.

7. Other Requirements

The project shall be developed using Agile methodology with sprint-based delivery (Sprint 0 – Setup, Sprint 1 – Backlog, Sprint 2 – Design, Sprint 3 – Core Implementation, Sprint 4 – Integration, Sprint 5 – Testing, Sprint 6 – Validation & Refinement), with GitHub (and Jira or a similar tool) actively used for user stories and backlog management, and CI/CD, code review, and basic security validation practised throughout, per the course guidelines (UE24CS341A). No database, internationalisation, or legal/regulatory requirements beyond local data storage and authorised-network-use are identified for this phase.

Appendix A: Glossary

Term	Definition
Zeek	An open-source network security monitoring engine used for protocol parsing, structured logging, and behavioural/anomaly detection.
.pcap	A standard file format for storing captured network packets, compatible with tools such as Wireshark.
conn.log / dns.log / http.log	Structured log files produced by Zeek summarising connection, DNS, and HTTP activity respectively.
Top talkers	The hosts on a network segment responsible for the largest volume of traffic or the highest number of connections.
Port-scan behaviour	A traffic pattern in which a single source probes multiple ports/hosts in quick succession, commonly associated with reconnaissance activity and flagged by Zeek's detection scripts.
Raspberry Pi	A single-board computer used as the primary hardware platform for the capture and analysis pipeline.
ESP32	A low-cost microcontroller with networking capability, used as an optional auxiliary hardware component where the final hardware design requires it.

Appendix B: Field Layouts

Sample field layout for a session record stored in the local session index:

Field	Length	Data Type	Description	Is Mandatory
Session ID	16	Alphanumeric	Unique identifier for the capture session	Y
Interface Name	20	String	Network interface used for the capture	Y
Start Timestamp	20	DateTime	Date and time the capture started	Y
End Timestamp	20	DateTime	Date and time the capture ended	N
Duration (sec)	10	Numeric	Total capture duration in seconds	N
Filter Applied	50	String	Basic protocol filter applied, if any	N
PCAP File Path	255	String	Local file path to the raw .pcap capture	Y
Zeek Log Path	255	String	Local file path to the associated Zeek logs	Y
Alert Count	6	Numeric	Number of alerts generated during the session	N
Export Status	15	String	Whether the session has been exported	N

Appendix C: Requirement Traceability Matrix

Sl. No	Requirement ID	Brief Description	Architecture Reference	Design Reference	Code File Reference	Test Case ID	System Test Case ID
1	REQ-1.1	Select network interface	Capture Layer / CaptureService	SAD 3.4, 4.2.1, 4.3.2	TBD (pending implementation)	UT-CAP-01	—
2	REQ-1.2	Start/stop capture session	Capture Layer / CaptureService	SAD 3.4, 4.2.1, 4.3.2	TBD (pending implementation)	IT-CAP-01	—
3	REQ-1.3	Optional maximum duration	Capture Layer / CaptureService	SAD 3.4, 4.2.1, 4.3.2	TBD (pending implementation)	UT-CAP-02	—
4	REQ-1.4	Optional protocol filter	Capture Layer / CaptureService	SAD 3.4, 4.2.1, 4.3.2	TBD (pending implementation)	IT-CAP-02	—
5	REQ-1.5	Stream packets to Zeek in real time	Zeek Integration Layer / LogReader	SAD 3.4, 4.2.1-4.2.3	TBD (pending implementation)	IT-CAP-01	—

Sl. No	Requirement ID	Brief Description	Architecture Reference	Design Reference	Code File Reference	Test Case ID	System Test Case ID
6	REQ-1.6	Write raw .pcap to storage	Capture Layer / Storage Layer	SAD 3.4, 4.2.1, 4.3.2	TBD (pending implementation)	IT-CAP-01, IT-STOR-01	—
7	REQ-1.7	Handle interface disconnect gracefully	Capture Layer / CaptureService	SAD 3.4, 4.2.1, 4.3.2	TBD (pending implementation)	—	ST-CAP-01
8	REQ-2.1	Read Zeek structured logs	Zeek Integration Layer / LogReader	SAD 3.4, 4.2.1-4.2.3	TBD (pending implementation)	IT-DASH-01	—
9	REQ-2.2	Protocol breakdown display	Python UI Layer / SummaryService	SAD 3.4, 4.2.2, 4.3.2	TBD (pending implementation)	UT-DASH-01	—
10	REQ-2.3	Top talkers display	Python UI Layer / SummaryService	SAD 3.4, 4.2.2, 4.3.2	TBD (pending implementation)	UT-DASH-02	—
11	REQ-2.4	Connection counts display	Python UI Layer / SummaryService	SAD 3.4, 4.2.2, 4.3.2	TBD (pending implementation)	UT-DASH-03	—
12	REQ-2.5	Real-time summary update	Python UI Layer / SummaryService	SAD 3.4, 4.2.2, 4.3.2	TBD (pending implementation)	IT-DASH-01	PERF-01
13	REQ-2.6	Simplified, understandable summary	Python UI Layer / SummaryService	SAD 3.4, 4.2.2, 4.3.2	TBD (pending implementation)	—	ST-USAB-01
14	REQ-3.1	Enable/disable alerting	Zeek Integration / AlertService	SAD 3.4, 4.2.3, 4.3.2	TBD (pending implementation)	IT-ALRT-02	—
15	REQ-3.2	Use Zeek detection scripts (port scan)	Zeek Integration / AlertService	SAD 3.4, 4.2.3, 4.3.2	TBD (pending implementation)	IT-ALRT-01	—
16	REQ-3.3	Generate alert entries (timestamp, source, reason)	Zeek Integration / AlertService	SAD 3.4, 4.2.3, 4.3.2	TBD (pending implementation)	UT-ALRT-01	—
17	REQ-3.4	Prioritised, readable alert list	Zeek Integration / AlertService	SAD 3.4, 4.2.3, 4.3.2	TBD (pending implementation)	UT-ALRT-02	—
18	REQ-3.5	No alerts beyond supported patterns	Zeek Integration / AlertService	SAD 3.4, 4.2.3, 4.3.2	TBD (pending implementation)	—	ST-ALRT-01
19	REQ-4.1	Maintain local session index	Storage Layer / SessionStore	SAD 3.4, 4.2.4, 4.3.2	TBD (pending implementation)	UT-STOR-01	—
20	REQ-4.2	Browse and select past sessions	Storage Layer / SessionStore	SAD 3.4, 4.2.4, 4.3.2	TBD (pending implementation)	UT-STOR-02	—
21	REQ-4.3	Export .pcap and/or Zeek logs	Storage Layer / SessionStore	SAD 3.4, 4.2.4, 4.3.2	TBD (pending implementation)	IT-STOR-01	—
22	REQ-4.4	Persist across restart/reboot	Storage Layer / SessionStore	SAD 3.4, 4.2.4, 4.3.2	TBD (pending implementation)	—	ST-STOR-01
23	REQ-4.5	Exported .pcap Wireshark-compatible	Storage Layer / SessionStore	SAD 3.4, 4.2.4, 4.3.2	TBD (pending implementation)	IT-STOR-01	—
24	NFR-01	Dashboard refresh within 2 s	Python UI Layer / SummaryService	SAD 3.4, 4.2.2, 4.3.2	TBD (pending implementation)	IT-DASH-01	PERF-01
25	NFR-02	5 Mbps with at most 1% loss	Capture / Zeek Integration Layers	SAD 3.7	TBD (pending implementation)	—	PERF-01
26	NFR-03	Disconnect handling, notify within 5 s	Capture Layer / CaptureService	SAD 3.4, 4.2.1, 4.3.2	TBD (pending implementation)	—	ST-CAP-01
27	NFR-04	Modules testable from .pcap	All layers (layered design)	SAD 3.5	TBD (pending implementation)	All UT/IT (pcap replay)	—
28	NFR-05	New user task within 5 minutes	Python UI Layer	SAD 4.5	TBD (pending implementation)	—	ST-USAB-01
29	SEC-01	No outbound transmission of data	Storage Layer / SessionStore	SAD 3.4, 4.2.4, 4.3.2	TBD (pending implementation)	—	ST-SEC-01
30	SEC-02	Owner-only file permissions	Storage Layer / SessionStore	SAD 3.4, 4.2.4, 4.3.2	TBD (pending implementation)	—	ST-SEC-02
31	SEC-03	Input validation / no injection	Validator / CaptureService	SAD 3.9, 4.2.1	TBD (pending implementation)	—	ST-SEC-03
32	SEC-04	Passive operation only	Capture Layer / CaptureService	SAD 3.4, 4.2.1, 4.3.2	TBD (pending implementation)	—	ST-SEC-04
33	SEC-05	Safe export path handling	Storage Layer / SessionStore	SAD 3.4, 4.2.4, 4.3.2	TBD (pending implementation)	—	ST-SEC-05

Part II - Software Architecture and Design Specification

Project: Pocket Packet Sniffer - Group 4 (Jackfruit Phase-1)

Version 1.0 | 6 October 2026

Revision History

Version	Date	Author	Change Summary
1.0	6 Oct 2026	Group 4	Initial draft: architecture, component diagram, 4 sequence diagrams, API design, security architecture.

Approvals

Role	Name	Signature/Date
Team Member	Ankita S (PES1UG24CS066)	
Team Member	Anusha Gupta (PES1UG24CS073)	
Team Member	Charan M (PES1UG24CS125)	
Team Member	Vaghasiya Akshar Arvindbhai (PES1UG25CS852)	
Course Coordinator		

1. Introduction

1.1 Purpose

This document specifies the software architecture and detailed design of the Pocket Packet Sniffer v1.0. It describes the component structure, the chosen architecture pattern and its rationale, the dynamic behaviour of the system's four core flows (as UML sequence diagrams), the module interface (API) design, error-handling strategy, and the security architecture that realise the objectives and requirements defined in the SRS (v1.0). It is the Design-phase deliverable that developers use to implement each owned module consistently.

1.2 Scope

Covers the internal software architecture of Pocket Packet Sniffer v1.0: the layered component structure (Presentation → Controller → Service → Engine & Data); sequence diagrams for Live Capture start, Real-Time Summary refresh, Alert generation and Session Export; the in-process API design for the Capture, Summary, Alert and Session Store services; and the security architecture addressing the security requirements proposed in the Test Plan (SEC-01 to SEC-05). Excludes the internal design of Zeek itself, Raspberry Pi/ESP32 hardware and firmware design, and Future Scope items (multiple simultaneous interfaces, onboard display, remote dashboard), which are out of scope per the Synopsis and SRS.

1.3 Audience

Primary audience: the four Group 4 developers (Ankita S, Anusha Gupta, Charan M, Vaghasiya Akshar Arvindbhai), each implementing the module they own. Secondary audience: course faculty and evaluators assessing architectural soundness, modularity and requirements traceability; and any future maintainer extending the device (e.g., adding an onboard display or remote dashboard).

1.4 Definitions

Acronyms and terms used here follow SRS Appendix A. Additional architecture-specific terms:

- **Controller:** the thin layer that receives user actions from the Python UI and dispatches them to the corresponding Service.
- **Service:** a module containing the business logic for one owned feature, with no direct terminal/UI I/O (supports the Maintainability and Testability attributes of SRS Section 6.4).

- LogReader: the Zeek Integration component that tails Zeek's structured logs (conn.log, dns.log, http.log, notice.log) and yields parsed records.
- SessionStore: the Storage Layer component that owns the session index and the session files on local disk.
- STRIDE: Spoofing, Tampering, Repudiation, Information disclosure, Denial of service, Elevation of privilege — the threat-modelling categories used in Section 3.9.

2. Document Overview

2.1 How to use this document

Each developer should read Section 3 (Architecture) before writing any code for their module, to confirm which layer their functions belong in and which other components they call. Before implementing a specific flow, read the matching sequence diagram and API table in Sections 4.2–4.3. Testers should read Sections 3.9 and 4.4 when designing security and negative test cases for the Test Plan. Evaluators can use Section 3.8 (Traceability to Requirements) to check that every SRS requirement has an architectural home.

2.2 Related Documents

- Software Requirements Specification (SRS), Pocket Packet Sniffer, Version 1.0 — source of all REQ-x.y IDs referenced throughout this document.
- Software Test Plan (STP), Pocket Packet Sniffer, Version 1.0 (Draft) — source of NFR-01..05 and the proposed SEC-01..05 IDs; consumes this document's component/interface design for test-case design and the RTM.
- Group 4 — Pocket Packet Sniffer Synopsis / Project Proposal.

3. Architecture

3.1 Goals & Constraints

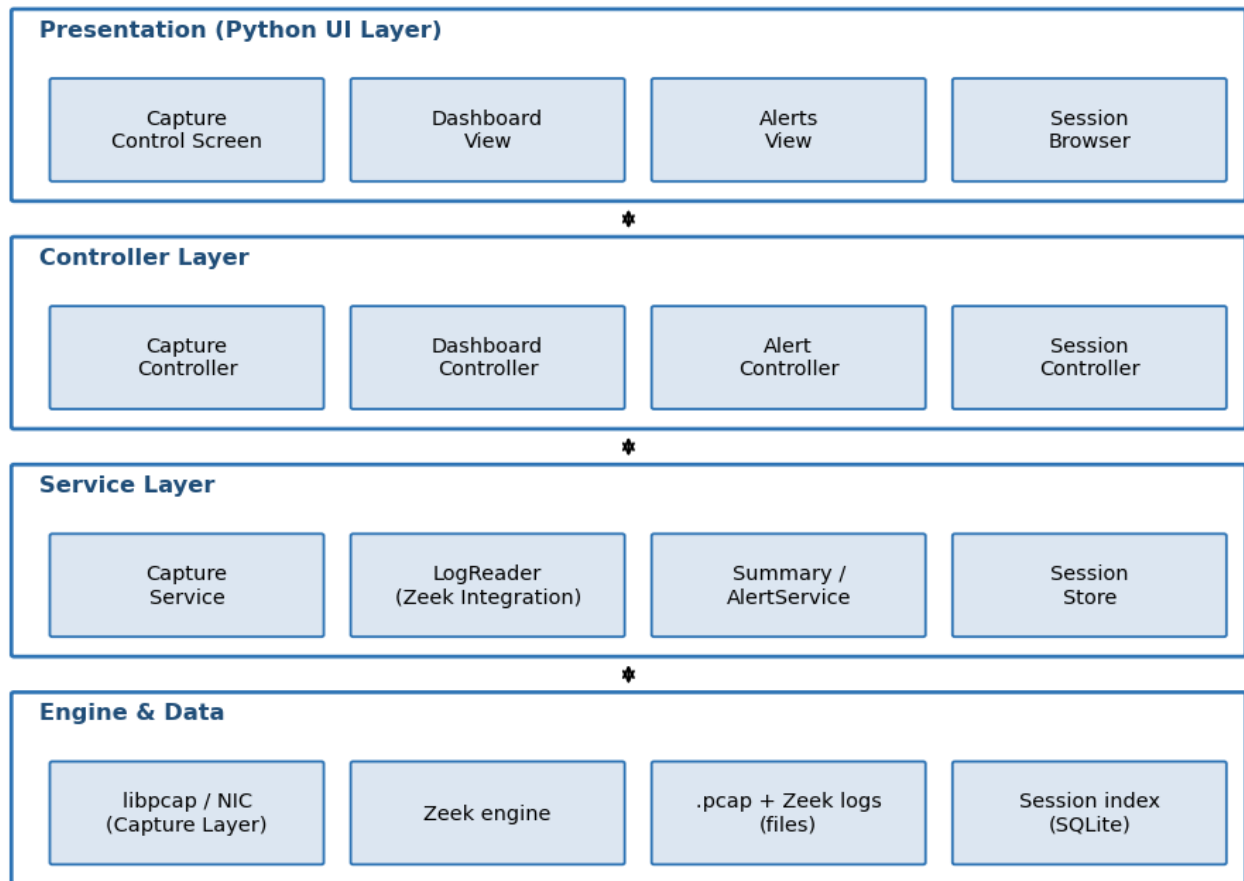
Goals: a modular, maintainable, independently-testable layered design in which each of the four owned features (Live Capture, Dashboard, Alerting, Session Storage/Export) can be implemented, unit-tested and reviewed in isolation using pre-recorded .pcap files (NFR-04); passive, local-only, secure-by-default behaviour; and a pipeline that keeps up with live traffic on a small LAN/lab segment with the dashboard refreshing at least every 2 seconds (NFR-01).

Constraints: packet analysis must be performed through Zeek rather than a custom parser; the control application and UI must be written in Python; raw captures must be stored in standard .pcap format; the target platform is Raspberry Pi-class Linux hardware with limited CPU, memory and storage; captured data stays on the device by default; one capture session per interface at a time (SRS Section 6.5); Agile, sprint-based delivery with Git/GitHub.

3.2 Stakeholders & Concerns

Stakeholder	Concerns
Students / TAs (end users)	Quick start of a capture, readable summary without Zeek/Wireshark expertise, trustworthy alerts, ability to keep and export sessions.
Junior network administrators	Low-effort first look at a segment; a device that does not disturb the network (passive operation) or crash on interface loss.
Course faculty / evaluators	Clean, modular design matching individual ownership; clear requirements-to-architecture-to-code traceability; demonstrable security validation.
Development team (Group 4)	Well-defined interfaces between the four owned modules so integration does not block anyone; unit testing without live network hardware.
Future maintainer	Ability to add an onboard display or remote dashboard without rewriting the Service and Storage layers (per Synopsis Future Scope).

3.3 Component (UML) Diagram



Cross-cutting: Validator (input checks) and Logger (application log)

Figure 3.1 — Pocket Packet Sniffer High-Level Component Diagram (Layered Architecture)

Two cross-cutting components are shared by every Service and omitted above for clarity: Validator (interface, filter, duration and export-path checks) and Logger (application log). Both are detailed in Section 3.4.

3.4 Component Descriptions

Component	Responsibility	Owner
Python UI (Capture screen, Dashboard, Alerts view, Session browser)	Renders the terminal/dashboard screens, reads user input, displays results and errors (SRS 3.1).	Shared (UI/UX consistency)
CaptureController / CaptureService	Interface listing, start/stop, duration and filter control, .pcap writing, streaming to Zeek, graceful interface-disconnect handling (REQ-1.1..1.7).	Ankita S
LogReader (Zeek Integration)	Starts/feeds Zeek and tails its structured logs incrementally for the Summary and Alert services (REQ-1.5, REQ-2.1, REQ-3.2).	Ankita S / Anusha Gupta
DashboardController / SummaryService	Aggregates protocol breakdown, top talkers and connection counts and serves a refreshed summary (REQ-2.1..2.6).	Anusha Gupta
AlertController / AlertService	Enables/disables alerting, activates Zeek detection scripts, builds prioritised alert entries with timestamp, source and reason (REQ-3.1..3.5).	Charan M
SessionController / SessionStore	Maintains the local session index, session files, browsing, retention across reboots and export (REQ-4.1..4.5).	Vaghasiya Akshar Arvindbhai
Validator	Validates interface names, filters, durations and export paths before any use (SEC-03, SEC-05).	Shared
Logger	Writes the application log (errors, warnings, session events) without logging captured payload content.	Shared
Zeek engine / libpcap	External engines: packet acquisition and .pcap output; protocol parsing, structured logs and detection scripts.	External (not developed)

Component	Responsibility	Owner
Local storage	Session directories holding .pcap files and Zeek logs, plus the session index (SQLite file) following SRS Appendix B fields.	Vaghasiya Akshar Arvindhbhai

3.5 Chosen Architecture Pattern and Rationale

Pattern: layered architecture (Presentation → Controller → Service → Engine/Data) arranged along a linear data pipeline (Capture → Zeek analysis → Summary/Alerts → Storage/Export), with one-directional, adjacent-layer-only calls.

Rationale: the layers and pipeline stages map one-to-one onto the team's product-ownership split (Synopsis Section 6), so each member can implement, unit-test and demo their Controller+Service pair independently. Keeping UI code separate from Services satisfies the SRS Maintainability and Testability attributes and lets Services run against pre-recorded .pcap files and sample Zeek logs with Zeek and the network interface mocked. Treating Zeek as an external engine behind the LogReader confines all Zeek-specific parsing to one place. A distributed or microservice style was considered and rejected: the product is a single-device, single-user tool on constrained Raspberry Pi hardware, so extra processes or a message bus would add CPU/memory load and complexity with no benefit.

3.6 Technology Stack & Data Stores

Layer / Concern	Technology	Notes
Language	Python 3	Control application and UI (SRS 2.5)
Packet capture	libpcap-based capture (e.g., tcpdump) run via argument-list subprocess	Writes standard .pcap; no shell string execution (SEC-03)
Analysis engine	Zeek (standard and extended scripts)	Produces conn.log, dns.log, http.log, notice.log
UI	Terminal-based Python interface (final choice per SRS 3.1)	Terminal or simple web/GUI dashboard allowed by SRS
Persistence	Session directories on local disk + SQLite session index	Fields per SRS Appendix B; local-only storage
Hardware	Raspberry Pi; optional ESP32 auxiliary component	Per Synopsis 3.3
Testing	pytest, pytest-cov, tcpreplay, nmap (lab traffic)	Per STP Section 6
Static analysis / CI	Bandit, SonarQube, Jenkins, Docker, GitHub	Per STP Section 6

3.7 Risks & Mitigations

Risk	Mitigation
Zeek is heavy for Raspberry Pi-class hardware and may drop packets under load	Single interface only; incremental log tailing; BPF filter support (REQ-1.4); replay-based performance test at 5 Mbps (NFR-02).
Interface disconnects or unplugs during capture	CaptureService monitors the capture process, finalises the partial .pcap, notifies the user within 5 seconds and returns to idle (REQ-1.7, NFR-03).
Command or path injection through filter, interface or export names	Validator allow-lists inputs; subprocesses use argument lists, never a shell; export names sanitised against path traversal (SEC-03, SEC-05).
Captured data exposure	Local-only storage, owner-only file permissions, no outbound transmission of captured data (SEC-01, SEC-02).
Alerts over-claim detection capability	Alerting restricted to Zeek-supported patterns; UI wording never presents the device as an IDS/SIEM (REQ-3.5).
Zeek unavailable or mismatched on the target OS	Sprint 0 spike to confirm installation; start-up check and clear fatal message (Section 4.4.3); tests use sample logs with Zeek mocked.

3.8 Traceability to Requirements

Architecture Component	Requirements Realised
CaptureController / CaptureService	REQ-1.1..1.7; NFR-03; SEC-03, SEC-04
LogReader (Zeek Integration)	REQ-1.5, REQ-2.1, REQ-3.2; NFR-04
DashboardController / SummaryService	REQ-2.1..2.6; NFR-01, NFR-02, NFR-05
AlertController / AlertService	REQ-3.1..3.5; SEC-04
SessionController / SessionStore	REQ-4.1..4.5; SEC-01, SEC-02, SEC-05
Validator / Logger	SEC-03, SEC-05; NFR-03

3.9 Security Architecture

Threats are analysed using STRIDE and mapped to the components responsible for mitigating each. The device is single-user with no login (SRS 6.3: access is controlled by physical/OS-level access), so the emphasis is on data confidentiality, input safety and passive operation.

STRIDE Category	Threat (Pocket Packet Sniffer-specific)	Mitigating Component(s) / Requirement
Spoofing	A crafted or replayed pcap/log is presented as genuine session data	Session records created only by SessionStore at capture end with generated Session ID; files stay in owner-only directories (SEC-02)
Tampering	Stored .pcap, Zeek logs or index altered by another OS user; malformed filter injected into the capture command	Owner-only permissions (SEC-02); Validator allow-list and argument-list subprocess (SEC-03)
Repudiation	Disputing when/on which interface a capture was made	Session index records interface, start/end timestamps and filter (SRS Appendix B); application log entries per session
Information Disclosure	Captured traffic leaks off the device or is read by other users; export written to an unintended path	No outbound transmission of captured data (SEC-01); owner-only files (SEC-02); sanitised export destination (SEC-05)
Denial of Service	Heavy traffic overwhelms the Pi; malformed input or interface loss crashes a session; active probing disrupts the segment	Single interface, filters, incremental log reading; graceful disconnect handling (REQ-1.7); passive-only operation (SEC-04)
Elevation of Privilege	Unsanitised input reaches a shell or Zeek with elevated capture privileges	Capture privileges limited to the capture subprocess; no shell execution; Validator checks before any call (SEC-03)

4. Design

4.1 Design Overview

The product is designed as four vertical, independently-ownable feature slices (Live Capture, Dashboard, Alerting, Session Storage/Export) sitting on a shared Zeek Integration and Storage foundation, each slice following the same Controller → Service shape described in Section 3. This section details the dynamic behaviour of the four core user-facing flows as UML sequence diagrams, followed by the module (API) interface design, the error-handling strategy, and the UX design.

4.2 UML Sequence Diagrams

Four sequence diagrams are provided — one per owned module — covering the highest-value flow in each (exceeding the minimum of two required).

4.2.1 Start Live Capture (Owner: Ankita S)

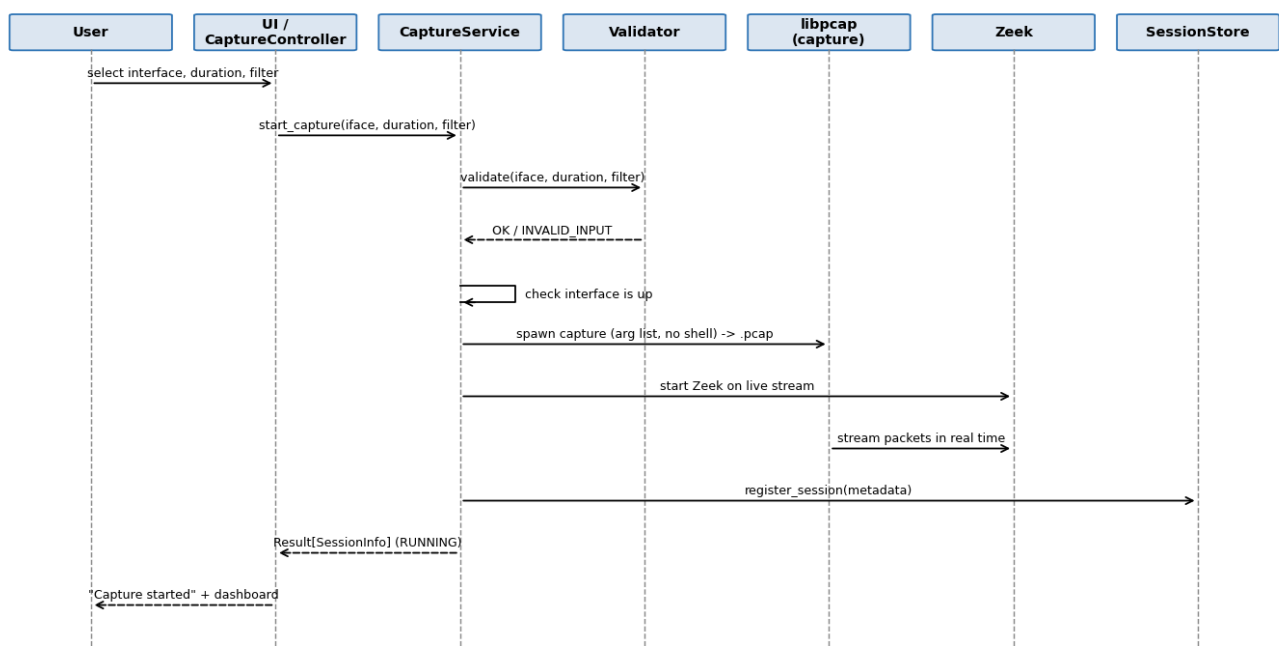


Figure 4.1 — Sequence Diagram: Start Live Capture (REQ-1.1..1.6)

CaptureService::start_capture() validates the interface, optional duration and optional filter through the Validator before anything is executed. The capture is launched with an argument list (no shell, SEC-03), writes the raw .pcap

to the session directory and streams packets to Zeek in real time. The session is registered with SessionStore as RUNNING. Stopping (user request or duration elapsed) finalises the .pcap and Zeek logs and marks the session COMPLETED; an interface disconnect finalises the partial .pcap and returns Status::INTERFACE_DISCONNECTED (REQ-1.7).

4.2.2 Real-Time Summary Refresh (Owner: Anusha Gupta)

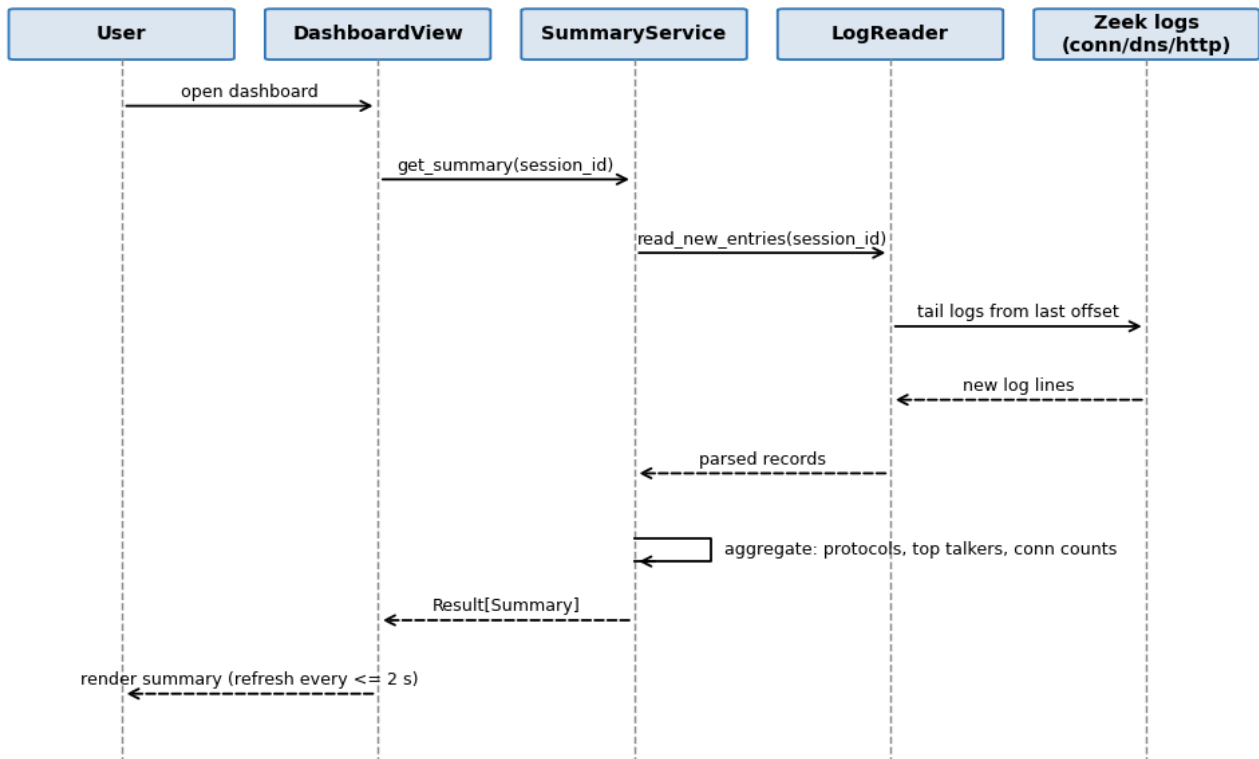


Figure 4.2 — Sequence Diagram: Real-Time Summary Refresh (REQ-2.1..2.6)

SummaryService does not re-read whole logs on each refresh: LogReader keeps a per-file offset and returns only new entries, which SummaryService merges into running counters (protocol breakdown, per-host byte/connection counts, total connections). The view refreshes at most every 2 seconds (NFR-01) and also works after the capture has ended, using the stored Zeek logs.

4.2.3 Alert Generation (Owner: Charan M)

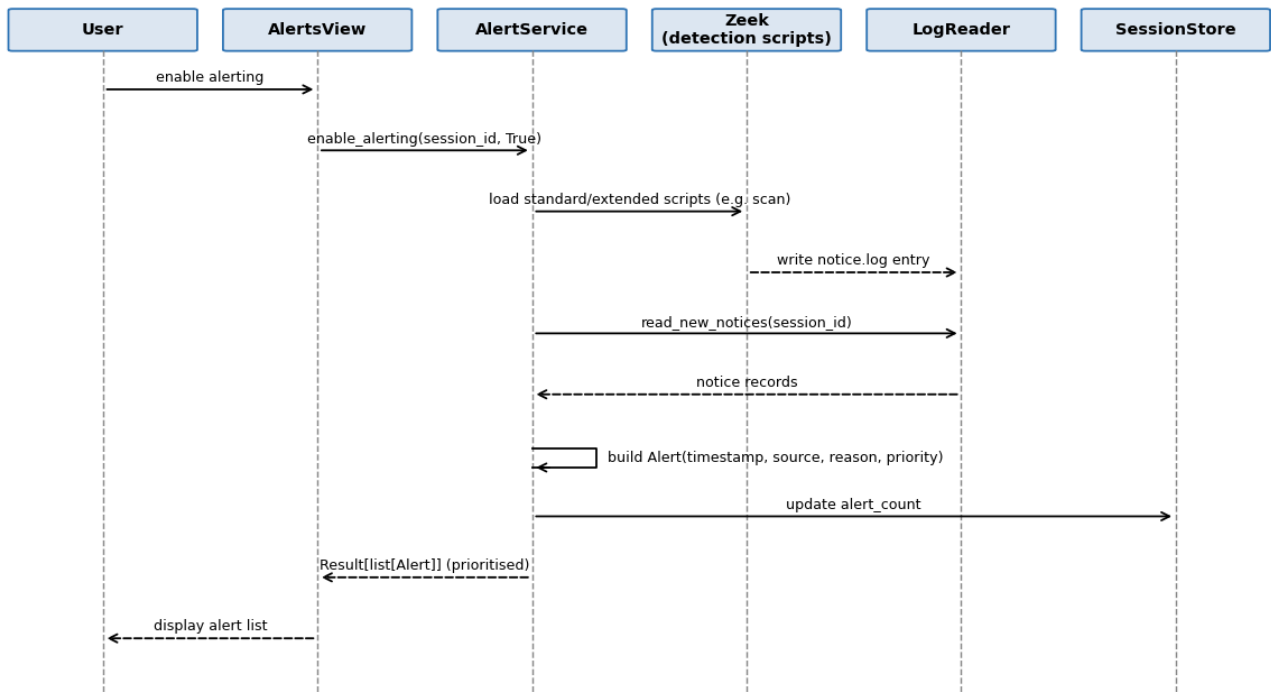


Figure 4.3 — Sequence Diagram: Alert Generation (REQ-3.1..3.5)

When alerting is enabled, Zeek runs with its standard/extended detection scripts (e.g., port-scan detection). AlertService reads Zeek's notice records through LogReader, converts each into an Alert with timestamp, source and reason, assigns a priority, sorts the list (highest priority first, then newest) and updates the session's alert count. Only patterns supported by Zeek or defined project-specific logic are reported (REQ-3.5).

4.2.4 Session Browse & Export (Owner: Vaghasiya Akshar Arvindbhai)

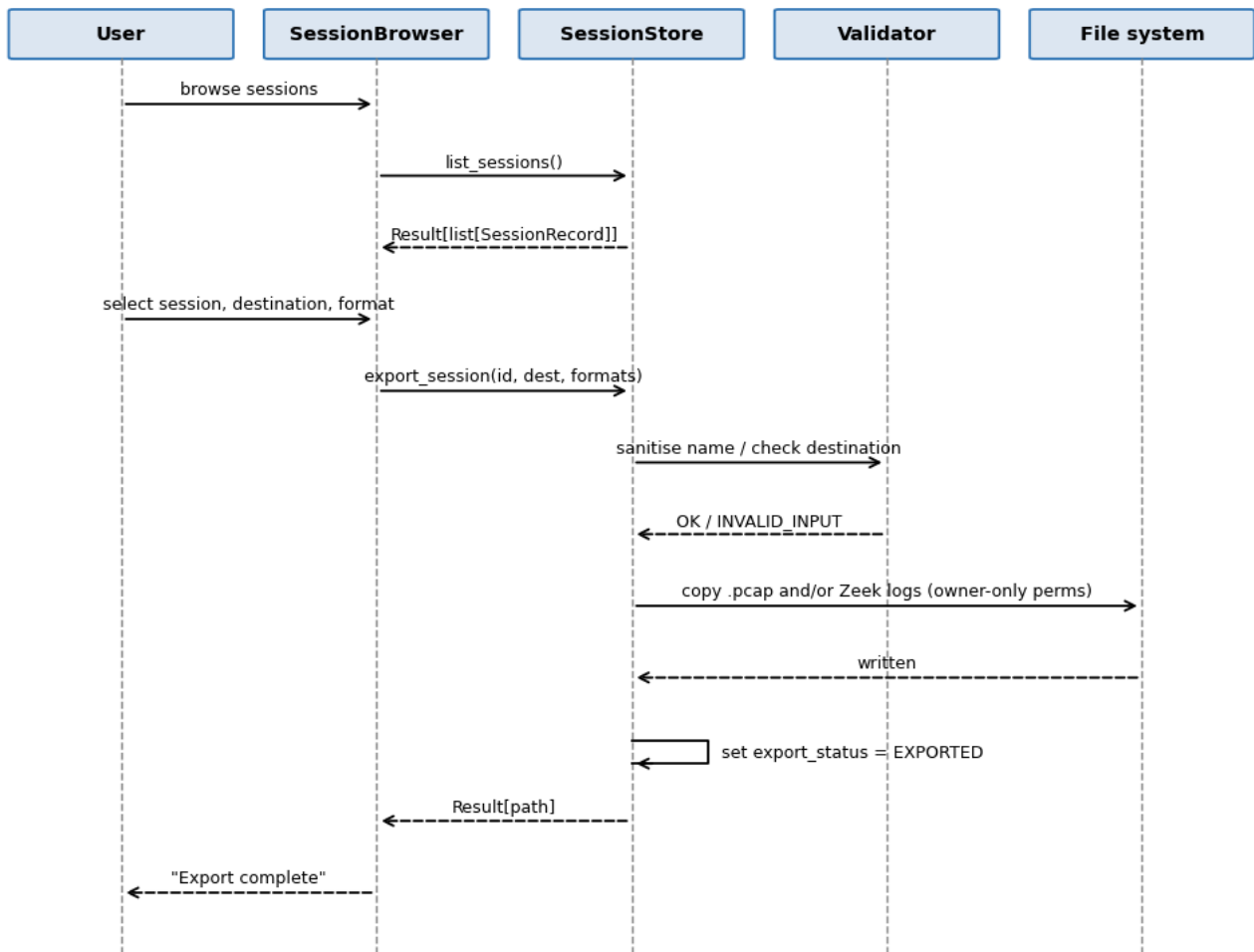


Figure 4.4 — Sequence Diagram: Session Browse & Export (REQ-4.1..4.5)

SessionStore reads the index to list sessions with date, duration and interface. On export, the session name and destination are sanitised against path traversal (SEC-05); the selected .pcap and/or Zeek logs are copied unmodified, so the exported .pcap stays Wireshark-compatible (REQ-4.5). Data persists across restarts and reboots because the index and files are on local disk (REQ-4.4).

4.3 API Design

The product is a single-process Python application with no mandatory network layer, so 'API' here means the in-process module interface (function signatures) between a Controller and its Service — not a REST/HTTP API. Interfaces are given for all four owned components (exceeding the minimum of two).

4.3.1 Shared Types (core/types.py)

Defined by Ankita S (Status/Result) and Vaghasiya Akshar Arvindbhai (SessionRecord) and adopted as the single canonical type set for every module, so all services return the same Status/Result shapes rather than each inventing its own error handling.

- class Status(Enum): OK, INVALID_INPUT, INTERFACE_UNAVAILABLE, INTERFACE_DISCONNECTED, ZEEK_ERROR, NOT_FOUND, PERMISSION_ERROR, IO_ERROR, INTERNAL_ERROR
- @dataclass Result[T]: status: Status, value: Optional[T], detail: str
- @dataclass SessionRecord: session_id, interface_name, start_ts, end_ts, duration_sec, filter_applied, pcap_path, zeek_log_path, alert_count, export_status — field lengths and mandatory flags per SRS Appendix B
- @dataclass Summary: protocol_breakdown: dict[str,int], top_talkers: list[(host, count)], connection_count: int, updated_at
- @dataclass Alert: timestamp, source, reason, priority (HIGH/MEDIUM/LOW)

4.3.2 Per-Component Interfaces

Component	Interface (signature)	Errors / Notes
CaptureService (Ankita S)	list_interfaces() -> Result[list[str]] start_capture(iface: str, duration: Optional[int], bpf_filter: Optional[str]) -> Result[SessionRecord] stop_capture(session_id: str) -> Status	INVALID_INPUT, INTERFACE_UNAVAILABLE, INTERFACE_DISCONNECTED, ZEEK_ERROR. Only one active session per interface (SRS 6.5).
SummaryService (Anusha Gupta)	get_summary(session_id: str) -> Result[Summary] reset(session_id: str) -> Status	NOT_FOUND for unknown session; an empty summary is a valid OK result, not an error.
AlertService (Charan M)	enable_alerting(session_id: str, enabled: bool) -> Status get_alerts(session_id: str) -> Result[list[Alert]]	ZEEK_ERROR if scripts cannot be loaded; zero alerts is a valid OK result.
SessionStore (Vaghasiya Akshar Arvindbhai)	register_session(rec: SessionRecord) -> Status list_sessions() -> Result[list[SessionRecord]] get_session(session_id: str) -> Result[SessionRecord] export_session(session_id: str, dest: str, formats: set[str]) -> Result[str]	NOT_FOUND, INVALID_INPUT (unsafe name/path), PERMISSION_ERROR, IO_ERROR.

4.4 Error Handling, Logging & Monitoring

Lead: Ankita S (capture errors). Storage/log angle: Vaghasiya Akshar Arvindbhai.

4.4.1 Error-Handling Strategy

- Detect early: user-supplied values are validated by Validator before any subprocess, Zeek or file operation (SEC-03).
- Return, don't crash: errors travel upward as Status values (Section 4.3.1); each layer handles an error or passes it up unchanged.
- Translate once: only the UI turns a Status into text, in the form "ERROR: <area>: <message>."; internal details (stack traces, file paths) are logged but not shown.
- Fail safe: on any capture failure the partial .pcap is finalised and kept readable, and the session is marked with its final state (NFR-03).
- Never crash the device: after start-up, every error returns the user to the previous screen (SRS 6.2, 6.4).

4.4.2 Error Catalogue

Status	Raised by	User message (UI)	Log level
INVALID_INPUT	Validator	ERROR: input: <field> is not valid.	WARN
INTERFACE_UNAVAILABL E	CaptureService	ERROR: interface: <name> is not available.	WARN
INTERFACE_DISCONNECTED	CaptureService	WARNING: interface lost; capture stopped, partial capture saved.	WARN
ZEEK_ERROR	LogReader / AlertService	ERROR: analysis: Zeek could not be started.	ERROR
NOT_FOUND	SessionStore / Services	ERROR: session: not found.	INFO
PERMISSION_ERROR	SessionStore	ERROR: file: permission denied.	ERROR
IO_ERROR	SessionStore	ERROR: file: could not write <name>.	ERROR
INTERNAL_ERROR	Service boundary catch	ERROR: internal: unexpected error, see log.	ERROR

4.4.3 Start-up / Fatal Conditions

Condition at start-up	Action	Exit code
Zeek not found or not runnable	Print "FATAL: Zeek not available.", log, exit	2
Session storage directory missing	Create it with owner-only permissions; continue	—
Session storage directory not writable	Print "FATAL: cannot write to session storage.", log, exit	3
Session index missing or unreadable	Create a new index; if corrupt, keep the corrupt file, warn, rebuild from session directories	—
Log file cannot be opened	Warn and continue, logging to stderr only	—

4.4.4 Capture & Storage Recovery Rules

- A capture process that exits unexpectedly is detected by CaptureService, which finalises the .pcap and marks the session INTERRUPTED so it still appears in history and can be exported.

- Session index updates are written atomically (single SQLite transaction), so a power loss on battery-powered operation never leaves a half-written index entry (REQ-4.4).
- Each session has its own directory; partial files are never deleted automatically.
- Zeek logs are read incrementally; if a log rotates or is truncated, LogReader resets its offset and continues.

4.4.5 Logging & Monitoring

The product keeps an application log (written by Logger across all modules, for diagnosing failures) separate from the per-session artefacts (.pcap, Zeek logs) and index. Captured payload content is never written to the application log. Monitoring in this version is CI- and log-based rather than live: the Jenkins/GitHub build and test pass rate is the team's health signal, and the dashboard shows the live capture state to the user.

4.5 UX Design

The Python interface follows SRS Section 3.1: a capture control screen (interface list, start/stop, duration, filter), a live dashboard (protocol breakdown, top talkers, connection counts), an alerts view (timestamp, source, reason, priority) and a session browser with export. Wording and layout are kept simple so a user with no Zeek or Wireshark experience can start a capture and find the top talker within 5 minutes (NFR-05). Every screen shows a clear title and current capture state, invalid input re-prompts rather than crashing, and every operation returns to the previous screen on completion or cancellation. UI/UX consistency across the four views is a shared team responsibility (Synopsis Section 6).

4.6 Open Issues & Next Steps

- Confirm whether the ESP32 is required by the final hardware design and, if so, define its interface to the capture layer (SRS 2.4).
- Finalise the UI form (terminal vs. simple web/GUI dashboard) in Sprint 2/3; the Controller/Service split is designed to make this a Presentation-layer change only.
- Decide the alert priority mapping for each supported Zeek notice type (e.g., port scan).
- Agree the measurable NFR-01..05 and SEC-01..05 values proposed in the Test Plan and add them to the SRS so SRS, STP and this document stay consistent.
- Benchmark Zeek on the target Raspberry Pi in Sprint 0 to confirm the 5 Mbps replay target (NFR-02).
- Future enhancements (out of v1.0 scope): multiple simultaneous interfaces, onboard LCD, remote dashboard, trend analysis across stored sessions.

5. Appendices

5.1 Glossary

See SRS Appendix A for product-level terms (Zeek, .pcap, conn.log/dns.log/http.log, top talkers, port-scan behaviour, Raspberry Pi, ESP32). Architecture-specific terms are defined in Section 1.4 of this document (Controller, Service, LogReader, SessionStore, STRIDE).

5.2 References

- Software Requirements Specification (SRS), Pocket Packet Sniffer, Version 1.0, Group 4.
- Software Test Plan (STP), Pocket Packet Sniffer, Version 1.0 (Draft), Group 4.
- Group 4 — Pocket Packet Sniffer Synopsis / Project Proposal.
- Zeek Network Security Monitor documentation — <https://zeek.org>
- OMG Unified Modeling Language (UML) Specification, Version 2.5.1.

5.3 Tools

- Diagramming: diagrams in this document were generated programmatically in a UML sequence-diagram convention; Draw.io/PlantUML are acceptable alternatives for future revisions.

- pytest and pytest-cov (unit testing and coverage), Bandit and SonarQube (static analysis), Jenkins and GitHub (CI/CD and version control), tcpreplay and nmap (test traffic).

Part III - Software Test Plan

Pocket Packet Sniffer - Group 4 (Jackfruit Phase-1)

Version 1.0 | 6 October 2026

1. Introduction

1.1 Purpose

This Software Test Plan defines the strategy, scope, environment, test cases and acceptance criteria used to verify that the Pocket Packet Sniffer satisfies the requirements in the SRS (v2.0) and conforms to the architecture in the SAD (v1.0). It is the source of the measurable NFR-01..05 and security SEC-01..05 requirements adopted into SRS v2.0.

1.2 Scope

In scope: Live Capture & Session Control (REQ-1.x), Real-Time Dashboard (REQ-2.x), Alerting (REQ-3.x), Session History, Storage & Export (REQ-4.x), and the NFR/SEC requirements. Out of scope: Zeek internals, ESP32/hardware firmware, and Future Scope items (multiple interfaces, onboard display, remote dashboard).

1.3 References

- SRS - Pocket Packet Sniffer, Version 2.0 (Part I).
- SAD - Pocket Packet Sniffer, Version 1.0 (Part II).
- Zeek documentation - <https://zeek.org>

2. Test Strategy

Testing follows the layered design: each Service is tested in isolation with Zeek and the network interface mocked (NFR-04), then integrated modules are tested against pre-recorded .pcap files replayed with tcpreplay, followed by system-level scenarios on Raspberry Pi hardware.

Level	Prefix	Focus	Approach
Unit	UT-	Single Service/Validator function behaviour	pytest + pytest-cov; mocked Zeek, sample logs
Integration	IT-	Controller-Service-LogReader-SessionStore interaction	Pre-recorded .pcap replay via tcpreplay; real Zeek
System	ST-	End-to-end behaviour, usability, security, reliability	Raspberry Pi device; lab traffic (nmap port scans)
Performance	PERF-	Throughput and refresh latency (NFR-01, NFR-02)	Replay of 5 Mbps stream on Pi

3. Test Environment and Tools

- Hardware: Raspberry Pi (target), optional ESP32; wired/wireless interface supporting promiscuous mode.
- Software: Python 3, Zeek, libpcap/tcpdump, SQLite, Wireshark (for export compatibility checks).
- Tools: pytest, pytest-cov, tcpreplay, nmap (lab traffic only), Bandit and SonarQube (static analysis), Jenkins, Docker, GitHub (CI/CD).
- Test data: pre-recorded benign .pcap files, port-scan .pcap generated in an authorised lab, malformed-input corpora.

4. Test Cases

Test ID	Verifies	Description	Expected Result
UT-CAP-01	REQ-1.1	List available interfaces; select valid and invalid interface	Valid interface accepted; invalid returns INTERFACE_UNAVAILABLE
UT-CAP-02	REQ-1.3	Start capture with max duration set	Session ends automatically at duration; state COMPLETED

Test ID	Verifies	Description	Expected Result
IT-CAP-01	REQ-1.2, 1.5, 1.6	Start, run and stop capture on replayed traffic	.pcap written, packets streamed to Zeek, files finalised on stop
IT-CAP-02	REQ-1.4	Apply a protocol/BPF filter during capture	Only matching traffic present in .pcap
ST-CAP-01	REQ-1.7, NFR-03	Disconnect interface mid-capture	No crash; user notified within 5 s; partial .pcap readable
UT-DASH-01	REQ-2.2	Protocol breakdown from sample conn.log	Counts per protocol correct
UT-DASH-02	REQ-2.3	Top talkers from sample logs	Hosts ranked by traffic/connections correctly
UT-DASH-03	REQ-2.4	Connection counts aggregation	Totals match log entries
IT-DASH-01	REQ-2.1, 2.5, NFR-01	Dashboard during live replay	New log entries reflected within 2 s; refresh at least every 2 s
ST-USAB-01	REQ-2.6, NFR-05	Novice user starts capture and finds top talker	Task completed in 5 minutes using on-screen guidance only
PERF-01	REQ-2.5, NFR-01, NFR-02	Replay 5 Mbps stream on Raspberry Pi	Packet loss at most 1%; dashboard lag within 2 s
IT-ALRT-01	REQ-3.2	Replay port-scan traffic with alerting enabled	Zeek port-scan notice detected
IT-ALRT-02	REQ-3.1	Enable/disable alerting for a session	Alerts generated only while enabled
UT-ALRT-01	REQ-3.3	Convert notice to alert	Alert contains timestamp, source and reason
UT-ALRT-02	REQ-3.4	Sort alerts	Highest priority first, then newest
ST-ALRT-01	REQ-3.5	Run traffic outside supported patterns	No alert raised; no over-claiming in UI wording
UT-STOR-01	REQ-4.1	Register completed session	Index record created with Appendix B fields
UT-STOR-02	REQ-4.2	List and select past sessions	Sessions listed with date, duration, interface; selection works
IT-STOR-01	REQ-4.3, 4.5, 1.6	Export .pcap and/or Zeek logs; open in Wireshark	Files exported unmodified; Wireshark opens .pcap
ST-STOR-01	REQ-4.4	Restart application and reboot device	All sessions and files retained
ST-SEC-01	SEC-01	Monitor outbound connections during capture/export	No outbound connection carries captured data
ST-SEC-02	SEC-02	Inspect permissions of session files and index	Owner-only (600/700 or stricter)
ST-SEC-03	SEC-03	Inject shell metacharacters in interface, filter, duration	Rejected as INVALID_INPUT; nothing reaches shell or Zeek; Bandit clean
ST-SEC-04	SEC-04	Observe segment during capture	No probe or injected packets transmitted
ST-SEC-05	SEC-05	Export with traversal names (e.g. ../)	Rejected or sanitised; written only to chosen destination

5. Requirements-to-Test Coverage

Every requirement in SRS Appendix C maps to at least one test case above: REQ-1.1..1.7 (UT/IT-CAP, ST-CAP-01), REQ-2.1..2.6 (UT/IT-DASH, ST-USAB-01, PERF-01), REQ-3.1..3.5 (IT/UT/ST-ALRT), REQ-4.1..4.5 (UT/IT/ST-STOR), NFR-01..05 (IT-DASH-01, PERF-01, ST-CAP-01, all UT/IT via pcap replay, ST-USAB-01) and SEC-01..05 (ST-SEC-01..05). The detailed matrix with architecture and design references is in SRS Appendix C.

6. Entry and Exit Criteria

- Entry: module implemented and reviewed on GitHub; test environment and sample .pcap files ready; build passes in Jenkins.
- Exit: all planned test cases executed; 100% of High-priority tests pass; no open critical/high defects; Bandit/SonarQube report no unresolved high-severity findings; coverage target agreed by the team met.

7. Schedule (aligned with Agile sprints)

Sprint	Testing Activity
Sprint 3 - Core Implementation	Unit tests (UT-) written alongside code
Sprint 4 - Integration	Integration tests (IT-) on replayed traffic
Sprint 5 - Testing	System, security, usability and performance tests (ST-, PERF-)
Sprint 6 - Validation and Refinement	Defect fixes, regression run, final sign-off

8. Roles and Responsibilities

Member	Test Ownership
Ankita S	Capture tests (UT/IT/ST-CAP), disconnect handling, NFR-03
Anusha Gupta	Dashboard tests (UT/IT-DASH), PERF-01, ST-USAB-01
Charan M	Alerting tests (UT/IT/ST-ALRT), security review of alert logic
Vaghasiya Akshar Arvindbhai	Storage/export tests (UT/IT/ST-STOR), ST-SEC-02, ST-SEC-05

9. Risks and Mitigations

Risk	Mitigation
Zeek too heavy on Raspberry Pi; packet drops	Early Sprint 0 benchmark; 5 Mbps replay test; BPF filters
Port-scan traffic cannot be tested on production networks	Use authorised lab environment with nmap and recorded .pcap files
Hardware not always available	Run unit/integration tests from pre-recorded .pcap (NFR-04)
Zeek unavailable on target OS	Verify installation in Sprint 0; mock Zeek in unit tests

10. Defect Management

Defects are logged as GitHub issues with severity (Critical/High/Medium/Low), the failing Test ID and the affected requirement, and are re-tested and regression-tested before closure.